Reinforcement Learning Techniques in Congestion Control

A Companion Guide to the 50 Most-Cited Studies

Gaurab Baral Nityashree Vangala

CS 6043: Networking
Fall 2026

Abstract

Our report on the 50 most-cited studies of reinforcement learning (RL) for congestion control names many learning methods (Q-learning, DQN, PPO, A3C, DDPG, TD3, IMPALA, imitation learning, offline RL, contextual bandits, multi-agent and multi-objective RL) without explaining them. This companion explains each one for a reader who knows networking but not RL. For every technique we give the idea, the update rule, the reason a congestion-control designer would pick it, and the studies among the 50 that use it, with the details those studies report. We close with the ideas that cut across algorithms: partial observability and recurrent policies, reward design, training curricula, hybrids with classic controllers, and verification of trained policies. A table at the end maps all 50 studies to their technique, action and reward.

Contents

1 How to read this guide **2**

2 RL vocabulary in congestion-control terms **2**
2.1 The Markov decision process . . . 2
2.2 Value functions and the Bellman equation . . . 3
2.3 Distinctions that recur . . . 4

3 Value-based methods **4**
3.1 Tabular Q-learning . . . 4
3.2 Function approximation with coarse coding . . . 5
3.3 Deep Q-Networks . . . 5

4 Policy-gradient methods **5**
4.1 REINFORCE . . . 5
4.2 Actor-critic and A3C . . . 6
4.3 Proximal Policy Optimization (PPO) . . . 6
4.4 IMPALA and asynchronous training . . . 6
4.5 Policy gradients through a known reward . . . 7

5 Off-policy actor-critic methods **7**
5.1 Deterministic policy gradient and DDPG . . . 7
5.2 TD3 . . . 7

6 Imitation learning **8**

7 Offline RL **8**

8 Bandits **8**

9 Multi-agent RL **9**

10 Multi-objective and multi-task RL **9**

11 Techniques that cut across algorithms **10**
11.1 Handling partial observability: history and recurrence 10
11.2 Reward design 10
11.3 Training curricula 10
11.4 Online learning with safeguards 11
11.5 Hybrids: learning on top of classic control 11

12 Tooling: training platforms, verification and deployment **11**

13 Comparing the families **12**

A Technique of each study **13**

1 How to read this guide

The companion report ranks the studies and discusses their design, testing and deployment. Here we cover only the learning methods. Section 2 introduces the vocabulary every later section relies on. Sections 3 to 5 cover the three families that train most of the controllers: value-based methods, policy-gradient methods and off-policy actor-critics. Sections 6 to 8 cover methods that learn from an expert, from logs, or from one-step feedback. Sections 9 and 10 cover settings with several agents or several objectives. Section 11 collects techniques that are used with any algorithm, and Section 12 covers the studies that analyze or run trained policies instead of training them. Section 13 compares the families, and Section A lists every study.

Where the labels come from. Each study's technique comes from our coding of its full text, abstract or our knowledge of the paper (the source of each is listed in the companion report). Of the 50 studies, 43 propose or train a controller. We assign each of them one primary family; Fig. 1 shows the counts. For 11 studies the source we coded says only "deep RL" and does not name an algorithm, and 8 studies are frameworks or analyses that train no controller of their own. Several studies combine families. MVFST-RL, for example, is a framework whose agent is trained with IMPALA, and ACC is a multi-agent system whose agents are DQNs. We discuss such studies in every section that applies.

2 RL vocabulary in congestion-control terms

2.1 The Markov decision process

RL models a controller as an *agent* that interacts with an *environment* in discrete steps [55]. At step t the agent observes a state s_t, picks an action a_t, receives a reward r_t, and the environment moves to s_{t+1}. A *Markov decision process* (MDP) assumes that s_{t+1} and r_t depend only on s_t and a_t, not on earlier history. In congestion control a step is usually one *monitor interval*, about one round-trip time (RTT) long:

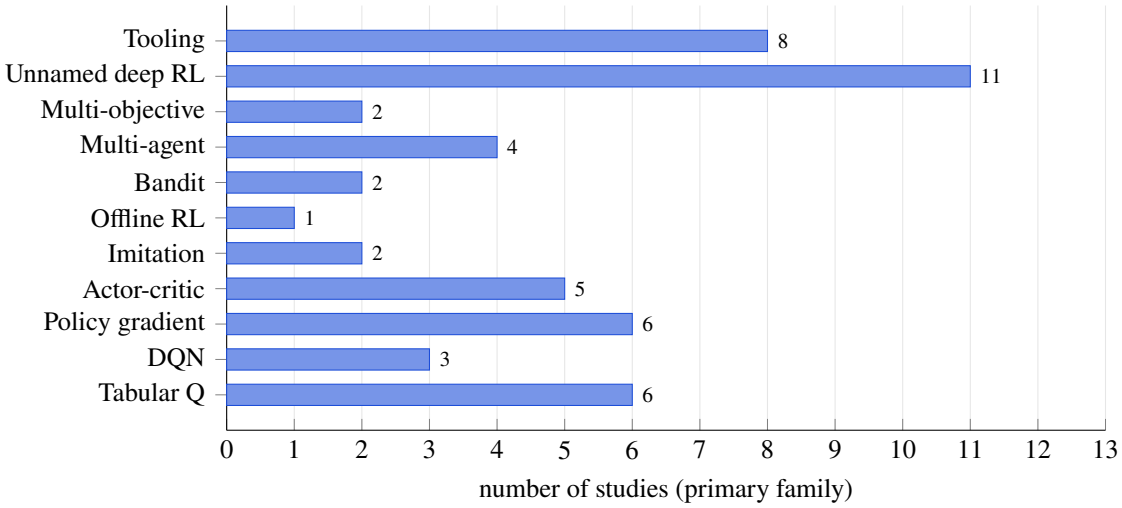


Figure 1: Primary technique family of the 50 studies. "Unnamed deep RL" ("Deep RL (n.s.)" in Section A) means the coded source does not name the algorithm. "Tooling" covers frameworks, verifiers and deployment methods.

- the **state** is a vector of recent statistics such as throughput, RTT, RTT gradient, loss rate and the current window;
- the **action** sets the congestion window (cwnd), the pacing rate, a video bitrate, a parameter of a classic algorithm, or, in a switch, a drop or ECN-marking decision;
- the **reward** scores the interval. Aurora's reward is $r_t = 10 \cdot \text{throughput} - 1000 \cdot \text{latency} - 2000 \cdot \text{loss}$ [19].

The agent's behavior is a *policy* π. A deterministic policy maps a state to an action, $a = \mu(s)$; a stochastic policy gives a probability for each action, $\pi(a \mid s)$. The goal is the policy that maximizes the expected *discounted return*

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}, \qquad 0 \le \gamma < 1. \tag{1}$$

The *discount factor* γ sets how far ahead the agent plans. With $\gamma = 0.99$ and one step per RTT, rewards roughly 100 RTTs ahead still matter. This matters in congestion control, because filling a queue now raises delay for many later intervals.

2.2 Value functions and the Bellman equation

Most algorithms estimate how good a state or action is. The *state value* $V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s]$ is the expected return from s when following π. The *action value* $Q^\pi(s,a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a]$ is the expected return after taking a in s and following π afterwards. Their difference $A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$ is the *advantage*: how much better a is than the policy's average action. The optimal action values satisfy the *Bellman optimality equation*

$$Q^*(s,a) = \mathbb{E}\big[\, r + \gamma \max_{a'} Q^*(s',a') \,\big|\, s, a \,\big], \tag{2}$$

and acting greedily, $a = \arg\max_a Q^*(s,a)$, is optimal. Learning methods that bootstrap from Eq. (2) use the *temporal-difference (TD) error* $\delta = r + \gamma \max_{a'} Q(s',a') - Q(s,a)$ as their learning signal.

2.3 Distinctions that recur

Discrete or continuous actions. A discrete action space is a short menu, for example "cwnd −10, −1, 0, +1, +10". Value-based methods need such a menu because they take a maximum over actions. A continuous action, such as a real-valued rate multiplier, needs a policy-gradient or actor-critic method. Of the coded design studies in Section A, the early Q-learning and DQN controllers use discrete actions, while most PPO, DDPG and TD3 controllers use continuous ones.

On-policy or off-policy. An *on-policy* method (REINFORCE, A3C, PPO) learns only from data generated by its current policy and discards that data after each update. An *off-policy* method (Q-learning, DQN, DDPG, TD3) can learn from data generated by any policy, including old versions of itself kept in a *replay buffer*. Off-policy methods reuse data and are more sample-efficient, which matters when each sample is a real RTT of a real flow.

Exploration. An agent must sometimes try actions it believes are worse, or it never learns that they are better. Value-based agents commonly use ε-*greedy* exploration (a random action with probability ε). Stochastic policies explore by sampling. Deterministic actor-critics add noise to the action. On a live network, exploration has a cost, since a random action can overflow a buffer, which is why OnRL and Loki add safeguards (Section 11.4).

Partial observability. A sender sees only its own ACKs. It cannot see the bottleneck queue or the competing flows, so the true state is hidden and the problem is a *partially observable* MDP (POMDP) [20]. Section 11.1 describes the two usual remedies: a history of observations in the state, or a recurrent network.

Function approximation. With a handful of discretized states, Q can be a table. With continuous measurements, the value function or policy is a parameterized function, usually a neural network with weights θ. "Deep RL" means RL with neural-network approximators.

3 Value-based methods

3.1 Tabular Q-learning

Q-learning [60] keeps a table $Q(s, a)$ and, after each transition (s, a, r, s'), moves the entry toward the TD target:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\big[\, r + \gamma \max_{a'} Q(s',a') - Q(s,a) \,\big], \tag{3}$$

where α is a step size. Q-learning is off-policy, because the target uses the greedy action $\max_{a'}$ whatever action the agent actually took next. With enough exploration and a decaying step size, it converges to Q^* in finite MDPs.

Why it fits congestion control. The table is tiny, inference is a lookup, and the algorithm needs no neural-network library. These properties suit kernels, routers and constrained IoT devices. The cost is that the state must be discretized coarsely (for example, three RTT levels times three throughput levels), so the controller cannot react to fine differences, and each state must be visited often before its entries are reliable.

3.2 Function approximation with coarse coding

When the state is continuous, a table needs either coarse bins or a huge number of cells. Coarse coding represents a state by a set of overlapping binary features and stores a weight per feature, so that $Q(s,a) = \sum_i w_{i,a}\,\phi_i(s)$ [55]. Nearby states share features and therefore share what they learn. *Tile coding* lays several offset grids (tilings) over the state space, and each active tile is a feature. *Kanerva coding* [21] instead places prototype points in the state space and activates the prototypes close to the current state. Both keep memory fixed and updates linear, which is why they appear in resource-constrained designs.

Used in the 50: RL-TCP [25] (tabular Q-learning over discretized TCP statistics, ns-2); QTCP [31] (Q-learning with generalized Kanerva coding, which adapts the prototype set during learning); SmartCC [30] (asynchronous Q-learning with hierarchical tile coding for multipath TCP); Li et al. [29] (Q-learning with function approximation to save memory on IoT devices); QRED [54] (Q-learning that tunes RED's maximum drop probability $\max_p$); Gómez et al. [14] (Q-learning plus a neural congestion predictor for ECN-based AQM). TCP-RL's group-based RL for the initial window [42] is closer to a bandit and is discussed in Section 8.

3.3 Deep Q-Networks

A Deep Q-Network (DQN) [39] replaces the table with a neural network $Q_\theta(s,a)$ that takes the state and outputs one value per discrete action. Training minimizes the squared TD error

$$L(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\Big[\big(r + \gamma \max_{a'} Q_{\theta^-}(s',a') - Q_\theta(s,a)\big)^2\Big]. \tag{4}$$

Naively combining Q-learning with a neural network is unstable, and DQN adds two fixes. The *replay buffer* $\mathcal{D}$ stores past transitions and samples mini-batches at random, which breaks the strong correlation between consecutive intervals of the same flow. The *target network* Q_{θ^-} is a copy of the network that is updated only every few thousand steps, so the target does not move with every gradient step. *Prioritized replay* [48] samples transitions with large TD error more often.

Why it fits congestion control. DQN handles continuous, high-dimensional observations (for example, a long window of RTT samples) while keeping a discrete action menu that is easy to bound and to implement in a kernel. Inference is one forward pass followed by an arg max.

Used in the 50: TCP-Drinc [63] (DQN whose network has convolutional and LSTM layers over a measurement history, ns-3); Owl [47] (DQN running in a Linux kernel module, with in-network signals added to end-to-end statistics); Kim et al. [24] (DQN for AQM in fog and edge IoT networks); ACC [67] (one DQN-style agent per switch that picks ECN marking thresholds, Section 9); DQL-MPTCP [44] (deep Q-learning combined with policy gradients and prioritized replay, Section 5); TCP-RL [42] (deep RL that selects one of 14 congestion-control algorithms).

4 Policy-gradient methods

4.1 REINFORCE

Policy-gradient methods parameterize the policy $\pi_\theta(a \mid s)$ directly and climb the gradient of the expected return $J(\theta)$. The policy-gradient theorem gives

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\big[\,\nabla_\theta \log \pi_\theta(a_t \mid s_t)\,\hat{A}_t\,\big], \tag{5}$$

where $\hat{A}_t$ estimates how good a_t was. REINFORCE [61] uses the observed return G_t, often minus a baseline such as the average return. Actions that led to above-average returns become more likely. For a continuous action the policy is usually a Gaussian whose mean (and sometimes variance) the network outputs, so the action can be any real number, such as a rate multiplier. Plain REINFORCE has high variance and needs many episodes. Iroko [46] includes it as one of its baseline agents.

4.2 Actor-critic and A3C

An *actor-critic* method learns two networks: the *actor* π_θ and a *critic* $V_\phi(s)$ that estimates state values. The critic supplies a lower-variance advantage, for example the one-step estimate $\hat{A}_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$ or the generalized advantage estimate (GAE) [49], which averages multi-step estimates. The critic is trained by regression on returns.

A3C (asynchronous advantage actor-critic) [40] runs many workers in parallel, each with its own copy of the environment. Workers compute gradients on their own experience and apply them asynchronously to shared parameters. Parallel workers decorrelate the data in the way a replay buffer does for DQN, and they shorten training wall time.

Used in the 50: QARC [18] (A3C chooses the video bitrate for real-time video, with a separate network that predicts video quality and feeds the agent); Fang et al. [12] (a PPO actor-critic for bandwidth estimation in real-time communication, below).

4.3 Proximal Policy Optimization (PPO)

A large policy-gradient step can collapse a policy that took hours to train. PPO [50] limits each update. It collects a batch of experience with the current policy $\pi_{\theta_{\text{old}}}$, forms the probability ratio $\rho_t(\theta) = \pi_\theta(a_t \mid s_t)/\pi_{\theta_{\text{old}}}(a_t \mid s_t)$, and maximizes the clipped objective

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t\Big[\min\big(\rho_t(\theta)\,\hat{A}_t,\ \text{clip}(\rho_t(\theta), 1-\epsilon, 1+\epsilon)\,\hat{A}_t\big)\Big], \tag{6}$$

with ϵ around 0.2. Once the new policy moves an action's probability more than ϵ away from the old one, the objective stops rewarding further movement. PPO can then take several gradient steps on each batch without destabilizing. It is simple to implement, works with discrete and continuous actions, and is the default choice of many RL libraries, which partly explains its popularity in networking.

Aurora as the worked example. Aurora [19] trains a PPO agent in a custom simulator. The state is a history of three scale-free statistics per monitor interval: the latency gradient, the latency ratio (current over minimum latency) and the sending ratio (packets sent over packets acknowledged). The action a_t is a real number that changes the sending rate x_t:

$$x_t = \begin{cases} x_{t-1}\,(1+\alpha a_t) & a_t \geq 0, \\ x_{t-1}/(1-\alpha a_t) & a_t < 0, \end{cases} \tag{7}$$

with a small scaling constant α, so that equal positive and negative actions cancel. The reward is the weighted sum given in Section 2. Because the inputs are ratios, the same policy applies across link speeds, which helped it generalize to bandwidths far above its training range [19].

Used in the 50: Aurora [19]; Genet [62] (Aurora's formulation, trained with a curriculum, Section 11.3); MOCC [37] (PPO conditioned on objective weights, Section 10); Fang et al. [12] (PPO for real-time-communication bandwidth estimation, reporting a gap between simulated and real reward); BoB [4] (PPO warm-started with a heuristic, Section 11.5); Iroko [46] (PPO as one of three agents for data-center traffic control); Libra [7] (a policy-gradient agent whose proposed rate competes with a classic algorithm's, Section 11.5).

4.4 IMPALA and asynchronous training

IMPALA [11] separates *actors*, which run the environment and record trajectories, from a central *learner*, which updates the network. Actors keep running with slightly stale copies of the policy, so the data they send is off-policy with respect to the learner's current weights. IMPALA corrects for this lag with *V-trace*, which

weights each step's TD error by a truncated importance ratio $\min(\bar{c}, \pi/\mu)$ between the learner policy π and the actor policy μ.

The separation matches a real network well. In a real stack the network does not pause while the agent computes, so an action decided now takes effect after a delay, and the sender keeps transmitting meanwhile. MVFST-RL [53] runs the policy asynchronously inside Facebook's QUIC implementation (mvfst) and trains it with IMPALA, treating the delay between observation and action as part of the problem.

4.5 Policy gradients through a known reward

When the reward is a known, differentiable function of the agent's output, the gradient can use that derivative instead of relying only on sampled returns. RL-CC [56] does this for data-center RDMA congestion control: its policy-gradient update uses the analytic derivative of the reward, and the problem is treated as multi-agent and partially observable, since each flow controls its own rate while sharing switches with the others. The reward is defined around the RTT inflation of each flow.

5 Off-policy actor-critic methods

5.1 Deterministic policy gradient and DDPG

For continuous actions a deterministic actor $\mu_\theta(s)$ can be trained by following the critic's gradient with respect to the action. The deterministic policy-gradient theorem [52] gives

$$\nabla_\theta J \approx \mathbb{E}_{s\sim\mathcal{D}}\big[\, \nabla_a Q_\phi(s,a)\big|_{a=\mu_\theta(s)}\, \nabla_\theta \mu_\theta(s) \,\big]. \tag{8}$$

In words, the critic tells the actor in which direction to nudge the action to increase value. DDPG [34] combines this with DQN's tricks: a replay buffer, slowly updated target networks for actor and critic, and exploration noise added to the action. The critic is trained on the TD target $r + \gamma Q_{\phi^-}(s', \mu_{\theta^-}(s'))$. Because DDPG is off-policy, it can learn from experience gathered by many distributed senders or by older policies.

5.2 TD3

DDPG's critic tends to overestimate values, and the actor then exploits those errors. TD3 [13] adds three changes. It learns two critics and uses the smaller of their estimates in the target, $y = r + \gamma \min_{i=1,2} Q_{\phi_i^-}(s', \tilde{a}')$. It smooths that target by adding clipped noise to the target action, $\tilde{a}' = \mu_{\theta^-}(s') + \mathrm{clip}(\mathcal{N}(0,\sigma), -c, c)$. It updates the actor and the target networks less often than the critics, typically once every two critic steps.

Orca as the worked example. Orca [1] keeps CUBIC running in the kernel on every ACK. Once per monitoring period, a TD3 agent in user space reads throughput, delay, loss and the current window and outputs $a \in [-1, 1]$, and the window is set to

$$\text{cwnd} \leftarrow 2^{2a} \cdot \text{cwnd}_{\text{CUBIC}}, \tag{9}$$

so the agent can scale CUBIC's window by a factor between $1/4$ and 4. Training uses many distributed actors in Mahimahi emulation that fill a shared replay buffer. The reward is based on *power*, a ratio of throughput to delay. Off-policy learning lets Orca reuse every actor's experience, and the deterministic output is a single number that the kernel can apply cheaply.

Used in the 50: Orca [1] (TD3 with distributed actors); Abbasloo et al.'s DeepCC [2] (a DDPG plug-in that caps the window of an existing TCP scheme to meet a delay target on cellular links); DRL-CC [65] (a deterministic policy-gradient actor-critic with an LSTM that sets the windows of all multipath TCP subflows); Naeem et al. [41] (an actor-critic with a fuzzy normalized neural network, run centrally in an SDN controller for IoT multipath traffic); DQL-MPTCP [44] (deep Q-learning combined with policy gradients, LSTM and prioritized replay for multipath window and traffic split); Iroko [46] (DDPG as one of its agents).

6 Imitation learning

Imitation learning trains a policy to copy an *expert* instead of maximizing a reward. In *behavioral cloning* the expert labels states with actions and the policy is trained by ordinary supervised learning on those pairs. Cloning has a known weakness: the learned policy makes small mistakes, drifts into states the expert never visited, and has no labels for them. DAgger [45] fixes this by iterating. It runs the current learned policy, asks the expert what it would have done in each visited state, adds those labels to the dataset and retrains. The training data then covers the states the learner actually reaches.

The expert can be of two kinds in congestion control. An *oracle* knows the simulated link's capacity and queue and can compute the ideal window, which is possible only in an emulator. An *existing heuristic* such as BBR or CUBIC can be imitated anywhere, but the learner then inherits the heuristic's limits unless it is refined further with RL.

Used in the 50: Indigo, released with Pantheon [66] (an LSTM policy trained to imitate an oracle that knows the ideal window of each emulated link, using DAgger-style iterations); Concerto [74] (imitation learning on more than one million logged video-telephony sessions to coordinate the video codec with the transport rate); Eagle [9] (deep RL with an LSTM that uses BBR as an expert during training and then matches or beats it).

7 Offline RL

Offline (or batch) RL learns a policy from a fixed dataset of transitions $\{(s, a, r, s')\}$ collected earlier by other policies, with no further interaction [27]. It promises the benefit of RL without the risk of exploring on a live network. The difficulty is *distribution shift*: an off-policy method such as DQN will happily assign high value to actions that never appear in the data, because nothing in the data contradicts its estimate, and the resulting policy then takes those untested actions. Offline RL algorithms therefore constrain the learned policy to stay near the actions in the dataset, or penalize value estimates for unseen actions, while still improving on the behavior that generated the data.

Unlike imitation learning, offline RL uses the reward. It can combine the best parts of several mediocre behaviors, so the learned policy can beat every policy in the dataset.

Used in the 50: Sage [69], which collects trajectories from a pool of existing heuristic controllers, learns a single policy from them offline, and in Internet and controlled tests outperforms the heuristics it observed. Concerto [74] also learns only from logged sessions, but by imitation rather than reward maximization.

8 Bandits

A *multi-armed bandit* is RL with a single step: the agent picks an arm (an action), receives a reward and the situation does not carry over to the next decision. The only difficulty is exploration, balancing arms that look good now against arms that have been tried too rarely. The *upper confidence bound* (UCB) rule picks the arm with the highest optimistic estimate, its mean reward plus a bonus that shrinks as the arm is tried more often.

A *contextual bandit* also observes a context vector x before choosing. LinUCB [28] assumes the expected reward of arm k is linear in the context, $\mathbb{E}[r \mid x, k] = \theta_k^\top x$, keeps a ridge-regression estimate $\hat{\theta}_k = A_k^{-1} b_k$ for

each arm, and picks

$$k^* = \arg\max_k \ \hat{\theta}_k^\top x + \alpha\sqrt{x^\top A_k^{-1} x}, \tag{10}$$

where the second term is large when the context is unlike those seen for arm k. After the reward arrives, it updates $A_k \leftarrow A_k + xx^\top$ and $b_k \leftarrow b_k + rx$. Each step is a few small matrix operations, cheap enough to run online.

Why it fits congestion control. Bandits suit decisions where the long-term effect of an action is small or can be ignored: which existing algorithm to use for the next period, or which initial window to give a new connection. Choosing among proven algorithms also keeps every action safe, since each arm is itself a deployed controller.

Used in the 50: Mutant [43] (an online LinUCB bandit that switches among existing kernel congestion-control protocols in real time, rewarded by power); TCP-RL [42] (group-based RL that picks the initial window for groups of users with similar network conditions, alongside deep RL that selects among 14 algorithms; it ran on a top global search engine's web servers for more than a year).

9 Multi-agent RL

Congestion control is naturally multi-agent: every flow, or every switch, makes its own decision, and each decision changes the environment the others see. Multi-agent RL (MARL) [73] studies how several learners interact. Three designs are common.

- **Independent learners.** Each agent treats the others as part of the environment. This is simple but non-stationary, because the other agents keep changing as they learn.
- **Parameter sharing.** All agents run copies of one policy and pool their experience. This suits homogeneous agents such as flows using the same protocol, or switches of the same type.
- **Centralized training with decentralized execution (CTDE).** During training, a critic sees global information (all flows' rates, the true queue); at run time each actor uses only local observations. MADDPG [35] is the best-known example.

The objective matters as much as the architecture. If each flow maximizes only its own reward, the learned policy tends to be greedy toward other copies of itself. Adding a fairness term or a shared reward makes the agents cooperate.

Used in the 50: Astraea [33] (several flows trained jointly in a faithful multi-flow environment, with local, global and historical observations and a reward that includes a fairness term; it reports near-optimal fairness and up to 8.4 times faster convergence); ACC [67] (one agent per data-center switch adjusting ECN thresholds, trained offline and then online, deployed on production clusters of 300 machines); RL-CC [56] (multi-agent, partially observable RDMA congestion control); He et al.'s DeepCC [16] (one agent per multipath TCP subflow, with *self-attention* [58] so that each agent weighs the states of the other subflows).

10 Multi-objective and multi-task RL

A scalar reward such as $10 \cdot \text{thr} - 1000 \cdot \text{lat} - 2000 \cdot \text{loss}$ fixes one trade-off. A video call wants low delay, and a bulk backup wants throughput. Multi-objective RL [15] keeps a reward *vector* $\mathbf{r} = (r_{\text{thr}}, r_{\text{delay}}, r_{\text{loss}})$ and a preference vector $\mathbf{w}$ that says how much each objective matters. The simplest approach, *linear scalarization*, optimizes $\mathbf{w}^\top \mathbf{r}$. A *preference-conditioned* policy $\pi(a \mid s, \mathbf{w})$ takes $\mathbf{w}$ as an extra input and is trained on many sampled preferences, so a single network covers the whole range of trade-offs and the application picks $\mathbf{w}$ at run time.

Used in the 50: MOCC [37] (a PPO policy with a preference sub-network that reads the objective weights; one model competes with the best specialized algorithm for each objective, and transfer learning lets it adapt to new objectives 14.2 times faster than prior work); AUTO [32] (multi-objective RL with a preference-adaptation model for satellite-ground networks).

Multi-task learning. A related idea trains one network on several tasks at once, so that shared layers learn features useful for all of them. Lei et al. [26] train a central agent in a software-defined network on congestion control with load balancing as an auxiliary task.

11 Techniques that cut across algorithms

11.1 Handling partial observability: history and recurrence

Because the sender cannot see the queue, a single interval's statistics do not identify the state. Two remedies appear. The first stacks the last k observations into the state vector, as Aurora does with its history of latency gradients and ratios. The second uses a recurrent network, usually an LSTM [17], whose hidden state summarizes the history of observations it has seen. An LSTM can in principle remember longer patterns than a fixed window, at the cost of harder training. Convolutional layers over a history window are a third option, used by TCP-Drinc.

Used in the 50: LSTMs in Indigo [66], DRL-CC [65], TCP-Drinc [63], Eagle [9] and DQL-MPTCP [44]; observation histories in Aurora [19], Genet [62], AUTO [32], BoB [4] and Astraea [33].

11.2 Reward design

The reward is the controller's specification. The 50 studies use four kinds.

- **Weighted sums** of throughput, delay and loss, as in Aurora, Genet and MOCC. The weights choose the operating point; a large delay weight produces a cautious controller.
- **Power**, throughput divided by delay, a classic measure of a good operating point used by Orca, Mutant and Gómez et al.
- **Utility functions** that combine several terms into one score, used by Owl, DRL-CC and Libra.
- **Application metrics**, such as quality of experience (bitrate, stalls, frame delay) for real-time video in QARC, Fang et al., Loki, BoB and HRCC, or flow completion time for TCP-RL and Xie et al.

Fairness is rarely in the reward. Astraea and IEACC [68] include it, and most other studies only measure it afterward.

11.3 Training curricula

Curriculum learning [3] orders training data, usually from easy to hard. Genet [62] applies this idea to network environments, with "hard" defined relative to rule-based baselines. It repeatedly searches the space of environment parameters (bandwidth, delay, queue size, loss) for settings where the current RL policy falls furthest behind a rule-based baseline, and adds those settings to training. Training then concentrates on the environments where the policy has the most to learn, rather than spreading effort over a uniform range where most environments are already solved. Genet's curriculum-trained policies beat both regularly trained RL policies and rule-based baselines, including in real environments.

11.4 Online learning with safeguards

Training in a simulator and freezing the policy leaves a gap to the real network. The alternative is to keep learning in deployment, which requires limiting the damage exploration can do.

Used in the 50: OnRL [70] (each user's device runs an agent that learns online in a mobile video service; the agents' experience is aggregated into one model, and a rule-based controller takes over when the learned policy behaves abnormally); Loki [71] (fuses a learned model with a reverse-engineered rule-based model through dual attention and trains online, evaluated in a commercial service over 101 million sessions); ACC [67] (offline pre-training, then online training on the switches).

11.5 Hybrids: learning on top of classic control

Of the 43 design studies, 15 keep a classic algorithm or safeguard in the deployed control loop. These hybrids can use any RL algorithm. What they share is a division of labor: the classic algorithm reacts at ACK timescale with well-understood behavior, and the learned part adjusts it more slowly or chooses when to override it. The patterns differ in how much authority the learner has.

- **Scale the classic output.** Orca's TD3 agent multiplies CUBIC's window (Section 5). Abbasloo et al.'s DeepCC caps the window of an existing scheme, and HRCC [59] sets a gain on a heuristic bandwidth estimate.
- **Tune a parameter.** QRED tunes RED's drop probability, and ACC tunes ECN thresholds.
- **Select among algorithms.** TCP-RL and Mutant choose which existing algorithm runs.
- **Compete and commit.** Libra [7] lets a classic algorithm and an RL policy each propose a rate, tries both briefly and commits to the one with higher utility.
- **Split timescales.** Spine [57] has a coarse DRL agent emit sub-policies that a lightweight executor applies at fine timescale, cutting control overhead by at least 80%.
- **Warm-start or fall back.** BoB [4] warm-starts PPO with a heuristic, and OnRL falls back to rules when the policy misbehaves.

12 Tooling: training platforms, verification and deployment

8 of the 50 studies train no controller of their own but shape how controllers are trained, checked or run.

Training platforms. Pantheon [66] provides emulators calibrated against real Internet paths and a common harness for comparing schemes. MVFST-RL [53] provides asynchronous IMPALA training inside a QUIC stack. OpenNetLab [10] combines a gym-style environment with distributed real nodes for real-time media. Iroko [46] emulates data-center topologies and compares PPO, REINFORCE and DDPG, and its authors report instability and overfitting to the training topology.

Formal verification. A neural policy is a function that can be checked mathematically. DNN verifiers in the line of Reluplex [22] encode a network with ReLU activations as constraints and ask a solver whether any input in a given range can produce a forbidden output, for example "the rate increases even though loss is high". Verily [23] applies such verifiers to deep-RL networking systems. Because an RL policy acts repeatedly, a property may only fail after several steps. whiRL 2.0 [8] therefore combines DNN verification with model checking over multiple steps, and it proves some requirements of trained policies, including Aurora's, while finding counterexamples for others.

Interpretation. Metis [38] explains deep-learning networking systems by converting a local policy into a decision tree (and a global, multi-component system into a hypergraph). A decision tree trained to mimic the network's outputs gives an operator readable rules and can be checked, debugged or even deployed in place of the network.

In-kernel inference. LiteFlow [72] addresses where a trained network runs. It places a compact snapshot of the neural network on a kernel fast path for low-latency inference, and keeps a user-space slow path that tunes the model and updates the kernel snapshot. This avoids the round trip between kernel and user space that a user-space agent would otherwise pay on every decision.

13 Comparing the families

Table 1 summarizes when each family fits. Three trends show in the 50 studies. The early work (2016–2019) used tabular Q-learning and DQN with discrete window steps. From Aurora onward, continuous-action methods (PPO, DDPG, TD3) dominate the most-cited controllers. The newest designs lean on prior knowledge, through imitation of heuristics (Eagle), offline RL on heuristic logs (Sage), bandits over existing protocols (Mutant) or explicit hybrids (Orca, Libra, Spine).

Table 1: Technique families in the 50 studies and the situations they fit.

Family	Actions	Fits when	Main risk
Tabular Q-learning	discrete	state can be coarsely discretized; memory or code size is tight (kernel, IoT, router)	coarse states; slow learning
DQN	discrete	rich observations, small action menu	unstable training; overestimation
PPO / A3C	discrete or continuous	a simulator gives plentiful on-policy data	sample-hungry; sensitive to reward scale
DDPG / TD3	continuous	data is costly and should be reused; many distributed actors	hyperparameter sensitivity; value overestimation (reduced by TD3)
IMPALA	either	acting and learning must run asynchronously, as in a real stack	stale-policy bias (corrected by V-trace)
Imitation learning	either	a good expert or oracle exists	cannot exceed the expert without further RL
Offline RL	either	logs of existing controllers are available; online exploration is unsafe	distribution shift to unseen actions
Contextual bandit	discrete	choosing among proven algorithms or settings; long-term effects small	ignores delayed consequences
Multi-agent RL	either	many flows or switches act together; fairness matters	non-stationarity; greedy equilibria
Multi-objective RL	either	applications need different trade-offs from one model	more training; preference coverage

A Technique of each study

Ranks follow the companion report. "Family" is the primary family used in Fig. 1; "n.s." means the coded source does not name the algorithm, and "n.r." that the reward is not reported there. Actions are marked continuous (cont.) or discrete (disc.) where we could code it. Rows coded from abstracts are lower bounds on what the papers contain.

#	Study	Family	Technique	Action	Reward
1	Aurora [19]	Policy gradient	PPO	rate (cont.)	throughput, delay, loss
2	Orca [1]	Actor-critic	TD3 (distributed actors)	cwnd (cont.)	power
3	Pantheon (Indigo) [66]	Tooling	imitation learning from emulator oracles (Indigo, LSTM)	–	–
4	QTCP [31]	Tabular Q	Q-learning + generalized Kanerva coding	cwnd (disc.)	throughput, delay
5	Genet [62]	Policy gradient	curriculum learning promoting environments where RL lags rule-based baselines	rate (cont.)	throughput, delay, loss
6	TCP-RL [42]	Bandit	group-based RL (IW) + deep RL (CCA selection among 14)	parameter; algorithm choice (disc.)	FCT, throughput
7	DRL-CC (MPTCP) [65]	Actor-critic	deterministic policy gradient actor-critic + LSTM	cwnd (cont.)	utility
8	OnRL [70]	Deep RL (n.s.)	online RL: per-user agents, aggregated model, safeguards against risky exploration	bitrate (cont.)	QoE
9	Metis [38]	Tooling	decision-tree and hypergraph interpretation of DL networking systems	–	–
10	Owl [47]	DQN	DQN in a Linux kernel module	cwnd (disc.)	utility
11	QARC [18]	Policy gradient	A3C + video-quality prediction network	bitrate (disc.)	QoE
12	SmartCC [30]	Tabular Q	Q-learning + hierarchical tile coding (asynchronous)	cwnd (disc.)	throughput
13	Sage [69]	Offline RL	offline RL learning from a pool of existing heuristics' trajectories	cwnd (cont.)	throughput, delay
14	Concerto [74]	Imitation	custom imitation learning on >1M logged sessions	bitrate (cont.)	–
15	whiRL 2.0 [8]	Tooling	DNN verification + model checking of DRL policies	–	–
16	Loki [71]	Deep RL (n.s.)	RL model fused with a reverse-engineered rule-based model (dual attention), trained online	bitrate (cont.)	QoE
17	RL-TCP [25]	Tabular Q	Q-learning	cwnd (disc.)	throughput, delay
18	Verily [23]	Tooling	formal verification of DRL policies	–	–
19	TCP-Drinc [63]	DQN	DQN with CNN+LSTM	cwnd (disc.)	throughput, delay
20	Eagle [9]	Imitation	DRL with LSTM guided by BBR as expert	rate	throughput, delay, loss

#	Study	Family	Technique	Action	Reward
21	DQL-MPTCP [44]	Actor-critic	deep Q-learning with policy gradients, LSTM, prioritized replay	cwnd; traffic split (cont.)	throughput, delay
22	MOCC [37]	Multi-objective	PPO with preference sub-network + transfer learning (user- and kernel-space)	rate (cont.)	throughput, delay, loss
23	DeepCC (Abbasloo et al.) [2]	Actor-critic	DDPG plug-in capping the cwnd of existing TCP	parameter (cont.)	delay
24	DeepCC (He et al., MPTCP) [16]	Multi-agent	multi-agent DRL with self-attention	cwnd	throughput
25	SDN actor-critic MPTCP for IoT [41]	Actor-critic	actor-critic with fuzzy normalized NN	cwnd (cont.)	n.r.
26	DRL initial window for 5G MEC [64]	Deep RL (n.s.)	DRL (+supervised refinement)	parameter	FCT, loss
27	RL BWE for RTC (Microsoft) [12]	Policy gradient	PPO (actor-critic)	bitrate (cont.)	QoE
28	RL-CC (Tessler et al.) [56]	Multi-agent	policy gradient with analytic reward derivative (multi-agent, partially observable)	rate (cont.)	other
29	Spine [57]	Deep RL (n.s.)	hierarchical: light CC executor + coarse-grained DRL sub-policy generator	parameter (cont.)	n.r.
30	Libra [7]	Policy gradient	classic and RL candidates compared in exploration/evaluation stages	rate (cont.)	utility
31	DQN AQM for fog/edge IoT [24]	DQN	DQN	drop/mark (disc.)	delay, throughput
32	iCoCoA [6]	Deep RL (n.s.)	DRL	parameter	n.r.
33	Astraea [33]	Multi-agent	multi-agent DRL with a faithful multi-flow training environment	rate (cont.)	throughput, delay, loss, fairness
34	MVFST-RL [53]	Tooling	IMPALA (asynchronous actor-learner, V-trace off-policy correction) with delayed actions	–	throughput, delay
35	AUTO [32]	Multi-objective	multi-objective RL + preference-adaptation model	rate (disc.)	throughput, delay
36	Iroko [46]	Tooling	PPO; REINFORCE; DDPG	–	utility, delay
37	Multi-task DRL CC (SDN) [26]	Deep RL (n.s.)	multi-task DRL	rate	n.r.
38	ACC [67]	Multi-agent	multi-agent DRL per switch (offline + online training)	ECN threshold (disc.)	throughput, delay
39	Q-learning TCP for memory-constrained IoT [29]	Tabular Q	Q-learning with function approximation	cwnd (disc.)	throughput, delay
40	OpenNetLab [10]	Tooling	platform for training/evaluating RL RTC CC	–	–
41	DRL age-of-information over TCP/IP [51]	Deep RL (n.s.)	deep RL	rate	other
42	BoB [4]	Policy gradient	PPO + heuristic warm start	bitrate (cont.)	QoE

#	Study	Family	Technique	Action	Reward
43	Mutant [43]	Bandit	online contextual bandit (LinUCB) over a pool of existing protocols	algorithm choice (disc.)	power
44	QRED [54]	Tabular Q	Q-learning tuning RED maxp	parameter (disc.)	other
45	LiteFlow [72]	Tooling	kernel fast path for NN inference + userspace slow path for tuning	rate	–
46	HRCC [59]	Deep RL (n.s.)	RL agent sets a gain on the heuristic (GCC-like) bandwidth estimate	parameter (cont.)	QoE
47	Intelligent AQM with ECN [14]	Tabular Q	Q-learning (+ANN congestion predictor)	drop/mark (disc.)	power
48	IEACC [68]	Deep RL (n.s.)	DRL with edge-aided congestion degrees	rate	throughput, loss, fairness
49	Hd-TCP [5]	Deep RL (n.s.)	DRL	cwnd	throughput, delay
50	AQM meets DRL [36]	Deep RL (n.s.)	DRL	drop/mark	n.r.

References

[1] S. Abbasloo, C.-Y. Yen, and H. J. Chao. Classic Meets Modern: a Pragmatic Learning-Based Congestion Control for the Internet. In *Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication*, pages 632–647. ACM, 2020. doi: 10.1145/3387514.3405892.

[2] S. Abbasloo, C.-Y. Yen, and H. J. Chao. Wanna Make Your TCP Scheme Great for Cellular Networks? Let Machines Do It for You! *IEEE Journal on Selected Areas in Communications*, 39(1):265–279, Jan. 2021. doi: 10.1109/jsac.2020.3036958.

[3] Y. Bengio, J. Louradour, R. Collobert, and J. Weston. Curriculum learning. In *Proceedings of the 26th International Conference on Machine Learning (ICML)*, pages 41–48, 2009.

[4] A. Bentaleb, M. N. Akcay, M. Lim, A. C. Begen, and R. Zimmermann. BoB: Bandwidth Prediction for Real-Time Communications Using Heuristic and Reinforcement Learning. *IEEE Transactions on Multimedia*, 25:6930–6945, 2023. doi: 10.1109/tmm.2022.3216456.

[5] L. Cui, Z. Yuan, Z. Ming, and S. Yang. Improving the Congestion Control Performance for Mobile Networks in High-Speed Railway via Deep Reinforcement Learning. *IEEE Transactions on Vehicular Technology*, 69(6):5864–5875, 2020. doi: 10.1109/tvt.2020.2984038.

[6] P. K. Donta, S. N. Srirama, T. Amgoth, and C. S. R. Annavarapu. iCoCoA: intelligent congestion control algorithm for CoAP using deep reinforcement learning. *Journal of Ambient Intelligence and Humanized Computing*, 14(3):2951–2966, Jan. 2023. doi: 10.1007/s12652-023-04534-8.

[7] Z. Du, J. Zheng, H. Yu, H. Zhang, and G. Chen. Libra: A Congestion Control Framework for Diverse Application Preferences and Network Conditions. *IEEE Transactions on Networking*, 33(2):510–525, Apr. 2025. doi: 10.1109/tnet.2024.3492096.

[8] T. Eliyahu, Y. Kazak, G. Katz, and M. Schapira. Verifying learning-augmented systems. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*, pages 305–318. ACM, Aug. 2021. doi: 10.1145/3452296.3472936.

[9] S. Emara, B. Li, and Y. Chen. Eagle: Refining Congestion Control by Learning from the Experts. In *IEEE INFOCOM 2020 - IEEE Conference on Computer Communications*, pages 676–685. IEEE, 2020. doi: 10.1109/infocom41043.2020.9155250.

[10] J. Eo, Z. Niu, W. Cheng, F. Y. Yan, R. Gao, J. Kardhashi, S. Inglis, M. Revow, B.-G. Chun, P. Cheng, and Y. Xiong. OpenNetLab: Open Platform for RL-based Congestion Control for Real-Time Communications. In *Proceedings of the 6th Asia-Pacific Workshop on Networking*, pages 70–75. ACM, 2022. doi: 10.1145/3542637.3542648.

[11] L. Espeholt, H. Soyer, R. Munos, K. Simonyan, V. Mnih, T. Ward, Y. Doron, V. Firoiu, T. Harley, I. Dunning, S. Legg, and K. Kavukcuoglu. IMPALA: Scalable Distributed Deep-RL with Importance Weighted Actor-Learner Architectures. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80, pages 1407–1416, 2018.

[12] J. Fang, M. Ellis, B. Li, S. Liu, Y. Hosseinkashi, M. Revow, A. Sadovnikov, Z. Liu, P. Cheng, S. Ashok, D. Zhao, R. Cutler, Y. Lu, and J. Gehrke. Reinforcement learning for bandwidth estimation and congestion control in real-time communications. arXiv:1912.02222, 2019.

[13] S. Fujimoto, H. van Hoof, and D. Meger. Addressing Function Approximation Error in Actor-Critic Methods. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80, pages 1587–1596, 2018.

[14] C. A. Gomez, X. Wang, and A. Shami. Intelligent Active Queue Management Using Explicit Congestion Notification. In *2019 IEEE Global Communications Conference (GLOBECOM)*, pages 1–6. IEEE, Dec. 2019. doi: 10.1109/globecom38437.2019.9013475.

[15] C. F. Hayes, R. Rădulescu, E. Bargiacchi, J. Källström, M. Macfarlane, M. Reymond, T. Verstraeten, L. M. Zintgraf, R. Dazeley, F. Heintz, E. Howley, A. A. Irissappane, P. Mannion, A. Nowé, G. Ramos, M. Restelli, P. Vamplew, and D. M. Roijers. A practical guide to multi-objective reinforcement learning and planning. *Autonomous Agents and Multi-Agent Systems*, 36(1), Apr. 2022. doi: 10.1007/s10458-022-09552-y. URL http://dx.doi.org/10.1007/s10458-022-09552-y.

[16] B. He, J. Wang, Q. Qi, H. Sun, J. Liao, C. Du, X. Yang, and Z. Han. DeepCC: Multi-Agent Deep Reinforcement Learning Congestion Control for Multi-Path TCP Based on Self-Attention. *IEEE Transactions on Network and Service Management*, 18(4):4770–4788, Dec. 2021. doi: 10.1109/tnsm.2021.3093302.

[17] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

[18] T. Huang, R.-X. Zhang, C. Zhou, and L. Sun. QARC: Video Quality Aware Rate Control for Real-Time Video Streaming based on Deep Reinforcement Learning. In *Proceedings of the 26th ACM international conference on Multimedia*, pages 1208–1216. ACM, Oct. 2018. doi: 10.1145/3240508.3240545.

[19] N. Jay, N. H. Rotman, P. B. Godfrey, M. Schapira, and A. Tamar. A Deep Reinforcement Learning Perspective on Internet Congestion Control. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, volume 97, pages 3050–3059, 2019.

[20] L. P. Kaelbling, M. L. Littman, and A. R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1-2):99–134, May 1998. doi: 10.1016/s0004-3702(98)00023-x. URL http://dx.doi.org/10.1016/S0004-3702(98)00023-X.

[21] P. Kanerva. *Sparse Distributed Memory*. MIT Press, 1988.

[22] G. Katz, C. Barrett, D. L. Dill, K. Julian, and M. J. Kochenderfer. Reluplex: An efficient SMT solver for verifying deep neural networks. In *Computer Aided Verification (CAV)*, pages 97–117, 2017.

[23] Y. Kazak, C. Barrett, G. Katz, and M. Schapira. Verifying Deep-RL-Driven Systems. In *Proceedings of the 2019 Workshop on Network Meets AI & ML - NetAI'19*, pages 83–89. ACM Press, 2019. doi: 10.1145/3341216.3342218.

[24] M. Kim, M. Jaseemuddin, and A. Anpalagan. Deep Reinforcement Learning Based Active Queue Management for IoT Networks. *Journal of Network and Systems Management*, 29(3), Apr. 2021. doi: 10.1007/s10922-021-09603-x.

[25] Y. Kong, H. Zang, and X. Ma. Improving TCP Congestion Control with Machine Intelligence. In *Proceedings of the 2018 Workshop on Network Meets AI & ML - NetAI'18*, pages 60–66. ACM Press, 2018. doi: 10.1145/3229543.3229550.

[26] K. Lei, Y. Liang, and W. Li. Congestion Control in SDN-Based Networks via Multi-Task Deep Reinforcement Learning. *IEEE Network*, 34(4):28–34, 2020. doi: 10.1109/mnet.011.1900408.

[27] S. Levine, A. Kumar, G. Tucker, and J. Fu. Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems. arXiv:2005.01643, 2020.

[28] L. Li, W. Chu, J. Langford, and R. E. Schapire. A contextual-bandit approach to personalized news article recommendation. In *Proceedings of the 19th International Conference on World Wide Web (WWW)*, pages 661–670, 2010.

[29] W. Li, F. Zhou, W. Meleis, and K. Chowdhury. Learning-Based and Data-Driven TCP Design for Memory-Constrained IoT. In *2016 International Conference on Distributed Computing in Sensor Systems (DCOSS)*, pages 199–205. IEEE, May 2016. doi: 10.1109/dcoss.2016.8.

[30] W. Li, H. Zhang, S. Gao, C. Xue, X. Wang, and S. Lu. SmartCC: A Reinforcement Learning Approach for Multipath TCP Congestion Control in Heterogeneous Networks. *IEEE Journal on Selected Areas in Communications*, 37(11):2621–2633, Nov. 2019. doi: 10.1109/jsac.2019.2933761.

[31] W. Li, F. Zhou, K. R. Chowdhury, and W. Meleis. QTCP: Adaptive Congestion Control with Reinforcement Learning. *IEEE Transactions on Network Science and Engineering*, 6(3):445–458, 2019. doi: 10.1109/tnse.2018.2835758.

[32] X. Li, F. Tang, J. Liu, L. T. Yang, L. Fu, and L. Chen. AUTO: Adaptive Congestion Control Based on Multi-Objective Reinforcement Learning for the Satellite-Ground Integrated Network. In *2021 USENIX Annual Technical Conference (USENIX ATC)*, 2021.

[33] X. Liao, H. Tian, C. Zeng, X. Wan, and K. Chen. Astraea: Towards Fair and Efficient Learning-based Congestion Control. In *Proceedings of the Nineteenth European Conference on Computer Systems*, pages 99–114. ACM, Apr. 2024. doi: 10.1145/3627703.3650069.

[34] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra. Continuous Control with Deep Reinforcement Learning. In *International Conference on Learning Representations (ICLR)*, 2016.

[35] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch. Multi-agent actor-critic for mixed cooperative-competitive environments. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 6379–6390, 2017.

[36] H. Ma, D. Xu, Y. Dai, and Q. Dong. An intelligent scheme for congestion control: When active queue management meets deep reinforcement learning. *Computer Networks*, 200:108515, Dec. 2021. doi: 10.1016/j.comnet.2021.108515.

[37] Y. Ma, H. Tian, X. Liao, J. Zhang, W. Wang, K. Chen, and X. Jin. Multi-objective congestion control. In *Proceedings of the Seventeenth European Conference on Computer Systems*, pages 218–235. ACM, Mar. 2022. doi: 10.1145/3492321.3519593.

[38] Z. Meng, M. Wang, J. Bai, M. Xu, H. Mao, and H. Hu. Interpreting Deep Learning-Based Networking Systems. In *Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication*, pages 154–171. ACM, 2020. doi: 10.1145/3387514.3405859.

[39] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, Feb. 2015. doi: 10.1038/nature14236. URL http://dx.doi.org/10.1038/nature14236.

[40] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu. Asynchronous Methods for Deep Reinforcement Learning. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, volume 48, pages 1928–1937, 2016.

[41] F. Naeem, G. Srivastava, and M. Tariq. A Software Defined Network Based Fuzzy Normalized Neural Adaptive Multipath Congestion Control for the Internet of Things. *IEEE Transactions on Network Science and Engineering*, 7(4):2155–2164, Oct. 2020. doi: 10.1109/tnse.2020.2991106.

[42] X. Nie, Y. Zhao, Z. Li, G. Chen, K. Sui, J. Zhang, Z. Ye, and D. Pei. Dynamic TCP Initial Windows and Congestion Control Schemes Through Reinforcement Learning. *IEEE Journal on Selected Areas in Communications*, 37(6):1231–1247, 2019. doi: 10.1109/jsac.2019.2904350.

[43] L. Pappone, A. Sacco, and F. Esposito. Mutant: Learning Congestion Control from Existing Protocols via Online Reinforcement Learning. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2025.

[44] S. R. Pokhrel and A. Walid. Learning to Harness Bandwidth With Multipath Congestion Control and Scheduling. *IEEE Transactions on Mobile Computing*, 22(2):996–1009, Feb. 2023. doi: 10.1109/tmc.2021.3085598.

[45] S. Ross, G. J. Gordon, and J. A. Bagnell. A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning. In *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 15, pages 627–635, 2011.

[46] F. Ruffy, M. Przystupa, and I. Beschastnikh. Iroko: A Framework to Prototype Reinforcement Learning for Data Center Traffic Control. arXiv:1812.09975, 2018.

[47] A. Sacco, M. Flocco, F. Esposito, and G. Marchetto. Owl: Congestion Control with Partially Invisible Networks via Reinforcement Learning. In *IEEE INFOCOM 2021 - IEEE Conference on Computer Communications*, pages 1–10. IEEE, May 2021. doi: 10.1109/infocom42981.2021.9488851.

[48] T. Schaul, J. Quan, I. Antonoglou, and D. Silver. Prioritized experience replay. In *International Conference on Learning Representations (ICLR)*, 2016.

[49] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations (ICLR)*, 2016.

[50] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal Policy Optimization Algorithms. arXiv:1707.06347, 2017.

[51] E. Sert, C. Sonmez, S. Baghaee, and E. Uysal-Biyikoglu. Optimizing age of information on real-life TCP/IP connections through reinforcement learning. In *2018 26th Signal Processing and Communications Applications Conference (SIU)*, pages 1–4. IEEE, May 2018. doi: 10.1109/siu.2018.8404794.

[52] D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, and M. Riedmiller. Deterministic policy gradient algorithms. In *Proceedings of the 31st International Conference on Machine Learning (ICML)*, pages 387–395, 2014.

[53] V. Sivakumar, O. Delalleau, T. Rocktäschel, A. H. Miller, H. Küttler, N. Nardelli, M. Rabbat, J. Pineau, and S. Riedel. MVFST-RL: An Asynchronous RL Framework for Congestion Control with Delayed Actions. arXiv:1910.04054, 2019.

[54] Y. Su, L. Huang, and C. Feng. QRED: A Q-Learning-based Active Queue Management Scheme. Journal of Internet Technology, 2018.

[55] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.

[56] C. Tessler, Y. Shpigelman, G. Dalal, A. Mandelbaum, D. Haritan Kazakov, B. Fuhrer, G. Chechik, and S. Mannor. Reinforcement Learning for Datacenter Congestion Control. *ACM SIGMETRICS Performance Evaluation Review*, 49(2):43–46, Jan. 2022. doi: 10.1145/3512798.3512815.

[57] H. Tian, X. Liao, C. Zeng, D. Sun, J. Zhang, and K. Chen. Efficient DRL-Based Congestion Control With Ultra-Low Overhead. *IEEE/ACM Transactions on Networking*, 32(3):1888–1903, 2024. doi: 10.1109/tnet.2023.3330737.

[58] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 5998–6008, 2017.

[59] B. Wang, Y. Zhang, S. Qian, Z. Pan, and Y. Xie. A Hybrid Receiver-side Congestion Control Scheme for Web Real-time Communication. In *Proceedings of the 12th ACM Multimedia Systems Conference*, pages 332–338. ACM, 2021. doi: 10.1145/3458305.3479970.

[60] C. J. C. H. Watkins and P. Dayan. Q-learning. *Machine Learning*, 8(3–4):279–292, 1992.

[61] R. J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3–4):229–256, 1992.

[62] Z. Xia, Y. Zhou, F. Y. Yan, and J. Jiang. Genet: automatic curriculum generation for learning adaptation in networking. In *Proceedings of the ACM SIGCOMM 2022 Conference*, pages 397–413. ACM, Aug. 2022. doi: 10.1145/3544216.3544243.

[63] K. Xiao, S. Mao, and J. K. Tugnait. TCP-Drinc: Smart Congestion Control Based on Deep Reinforcement Learning. *IEEE Access*, 7:11892–11904, 2019. doi: 10.1109/access.2019.2892046.

[64] R. Xie, X. Jia, and K. Wu. Adaptive Online Decision Method for Initial Congestion Window in 5G Mobile Edge Computing Using Deep Reinforcement Learning. *IEEE Journal on Selected Areas in Communications*, 38(2):389–403, Feb. 2020. doi: 10.1109/jsac.2019.2959187.

[65] Z. Xu, J. Tang, C. Yin, Y. Wang, and G. Xue. Experience-Driven Congestion Control: When Multi-Path TCP Meets Deep Reinforcement Learning. *IEEE Journal on Selected Areas in Communications*, 37(6): 1325–1336, 2019. doi: 10.1109/jsac.2019.2904358.

[66] F. Y. Yan, J. Ma, G. D. Hill, D. Raghavan, R. S. Wahby, P. Levis, and K. Winstein. Pantheon: The Training Ground for Internet Congestion-Control Research. In *2018 USENIX Annual Technical Conference (USENIX ATC)*, 2018.

[67] S. Yan, X. Wang, X. Zheng, Y. Xia, D. Liu, and W. Deng. ACC: automatic ECN tuning for high-speed datacenter networks. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*, pages 384–397. ACM, Aug. 2021. doi: 10.1145/3452296.3472927.

[68] J. Yang, Y. Chen, K. Xue, J. Han, J. Li, D. S. L. Wei, Q. Sun, and J. Lu. IEACC: An Intelligent Edge-Aided Congestion Control Scheme for Named Data Networking With Deep Reinforcement Learning. *IEEE Transactions on Network and Service Management*, 19(4):4932–4947, Dec. 2022. doi: 10.1109/tnsm.2022.3196344.

[69] C.-Y. Yen, S. Abbasloo, and H. J. Chao. Computers Can Learn from the Heuristic Designs and Master Internet Congestion Control. In *Proceedings of the ACM SIGCOMM 2023 Conference*, pages 255–274. ACM, 2023. doi: 10.1145/3603269.3604838.

[70] H. Zhang, A. Zhou, J. Lu, R. Ma, Y. Hu, C. Li, X. Zhang, H. Ma, and X. Chen. OnRL: improving mobile video telephony via online reinforcement learning. In *Proceedings of the 26th Annual International Conference on Mobile Computing and Networking*, pages 1–14. ACM, 2020. doi: 10.1145/3372224.3419186.

[71] H. Zhang, A. Zhou, Y. Hu, C. Li, G. Wang, X. Zhang, H. Ma, L. Wu, A. Chen, and C. Wu. Loki: improving long tail performance of learning-based real-time video adaptation by fusing rule-based models. In *Proceedings of the 27th Annual International Conference on Mobile Computing and Networking*, pages 775–788. ACM, Oct. 2021. doi: 10.1145/3447993.3483259.

[72] J. Zhang, C. Zeng, H. Zhang, S. Hu, and K. Chen. LiteFlow: towards high-performance adaptive neural networks for kernel datapath. In *Proceedings of the ACM SIGCOMM 2022 Conference*, pages 414–427. ACM, Aug. 2022. doi: 10.1145/3544216.3544229.

[73] K. Zhang, Z. Yang, and T. Başar. *Multi-Agent Reinforcement Learning: A Selective Overview of Theories and Algorithms*, pages 321–384. Springer International Publishing, 2021. ISBN 9783030609900. doi: 10.1007/978-3-030-60990-0_12. URL http://dx.doi.org/10.1007/978-3-030-60990-0_12.

[74] A. Zhou, H. Zhang, G. Su, L. Wu, R. Ma, Z. Meng, X. Zhang, X. Xie, H. Ma, and X. Chen. Learning to Coordinate Video Codec with Transport Protocol for Mobile Video Telephony. In *The 25th Annual International Conference on Mobile Computing and Networking*, pages 1–16. ACM, Oct. 2019. doi: 10.1145/3300061.3345430.